Lecture 10 Scribe Exam

L10_S2.pdf
PDF

"Generate a lecture scribe suitable for exam preparation based ONLY on the provided context (Lecture 10 slides). The target audience is a student preparing for a closed-notes exam.

Instructions:

Reason Step-by-Step: While writing the scribe, explicitly explain the reasoning behind definitions, assumptions, proofs, and example solutions exactly as presented in the provided context. Include this reasoning directly in the scribe text to help the student understand 'why' and 'how'.

Faithfulness: Include definitions, notation, theorem statements, and worked examples exactly as they appear in the slides. Do not hallucinate or add information from outside the context.

Logical Flow: Make the logical dependencies between concepts clear.

Structure: Organize the content with clear headings and subheadings corresponding to the lecture flow.

Goal: The output should allow a student to recall definitions, reconstruct proofs, and follow example solutions step-by-step."

Below is a **strict, exam-ready lecture scribe for Lecture 10**, written **only from the provided Lecture 10 slides** and **faithful to their definitions, notation, order, and explanations**.
This scribe is designed to be a **sole reference for a closed-notes exam**, enabling recall of definitions, reconstruction of reasoning, and step-by-step understanding of the algorithms exactly as presented.

All content is derived from the uploaded slides .

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

We use the **min-cut algorithm** in various applications to solve problems related to:

- **Network connectivity**
- **Reliability**
- **Optimization**

The lecture highlights the following applications:

1. **Network Design**
   Min-cut helps improve the efficiency of communication and optimize network flow.
   The algorithm is used to find the **minimum capacity cut**, which identifies bottlenecks in a network.
2. **Communication Networks**
   Min-cut is useful for understanding the **vulnerability of networks to failures**. By finding minimum cuts, we can design more **robust and fault-tolerant networks**.
3. **VLSI Design**
   In **Very Large Scale Integration (VLSI)** design, min-cut is used to partition circuits into smaller components, reducing **interconnectivity complexity**.

1.2 What Is Min-Cut?

Definition: Cut-Set
A **cut-set** in a graph is defined as:

> A set of edges whose removal breaks the graph into two or more connected components.

This definition explains *why* cuts are useful: removing these edges disconnects the graph, revealing weak points in connectivity.

Definition: Minimum Cut (Min-Cut)
Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut problem** is:

> To find a cut-set in G with **minimum cardinality**.

That is, among all possible edge sets that disconnect the graph, we want the one with the **smallest number of edges**.

Observation on Randomized Algorithms
Min-cut algorithms such as **Karger's algorithm** are **randomized** and **sensitive to the initial choice of edges**.

- If the algorithm contracts **critical edges early**, it may affect the final cut.
- This explains why some runs succeed while others fail, even on the same graph.

1.3 Edge Contraction (Core Operation)

The **main operation** in the min-cut algorithm is **edge contraction**.

Definition: Edge Contraction
Edge contraction is an operation that:

> Removes an edge from the graph while **simultaneously merging the two vertices** connected by that edge.

Detailed Explanation
When contracting an edge (u, v):

1. Vertices u and v are **merged into a single vertex**.
2. **All edges connecting u and v** are eliminated.
3. **All other edges** in the graph are retained.
4. The resulting graph:

- **May have parallel edges**
- **Will not have self-loops**

This operation reduces the size of the graph while preserving cut information.

---

2. Successful and Unsuccessful Min-Cut Runs

2.1 Successful Min-Cut Run

A **successful min-cut run** refers to:

> A run in which the algorithm correctly identifies the **minimum cut** of the graph.

The illustrated sequence of contractions avoids contracting edges that belong to the true minimum cut until the end.

2.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to:

> An iteration where the algorithm **fails** to correctly identify the minimum cut.

This happens when:

- A **minimum-cut edge is contracted too early**
- The structure of the true minimum cut is destroyed during contraction

This distinction motivates using **multiple randomized runs**.

---

3. Max-Flow Min-Cut Theorem

3.1 Theorem Statement

The **Max-Flow Min-Cut Theorem** states:

> *In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.*

This theorem links two seemingly different problems:

- Maximizing flow
- Minimizing cut capacity

3.2 Supporting Definitions

- **Capacity of a cut**
  The sum of the capacities of edges oriented from a vertex in set X to a vertex in set Y.
- **Minimum cut**
  The cut with the **smallest possible capacity**.
- **Minimum cut capacity**
  The capacity value of the minimum cut.
- **Maximum flow**
  The **largest possible flow** from source S to sink T.

The theorem asserts that these two quantities are equal.

---

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Algorithm

Theorem Used
Let s and t be two vertices of graph G.
Let $G/\{s, t\}$ be the graph obtained by merging s and t.

Then:

> A minimum cut of G is the **smaller** of:

- A minimum $s-t$ cut of G, and
- A minimum cut of $G/\{s, t\}$

Reasoning
- If a minimum cut **separates s and t**, it is captured by the minimum $s-t$ cut.
- Otherwise, merging s and t preserves the minimum cut, which is found in the contracted graph.

4.2 Pseudocode: Deterministic Min-Cut

Algorithm 1: `MinimumCutPhase(G, a)`

1. Initialize $A \leftarrow \{a\}$

2. While $A \neq V$:
    - Add to A the **most tightly connected vertex**
3. Return the cut weight as the **cut of the phase**

Algorithm 2: `MinimumCut(G)`

1. While $|V| \geqslant 1$:
    - Choose any $a \in V$
    - Run `MinimumCutPhase(G, a)`
    - If the cut-of-the-phase is lighter than the current minimum cut:
        - Store it
    - Shrink G by merging the last two vertices added
2. Return the minimum cut

This algorithm guarantees an **exact** minimum cut.

---

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithms?

Randomized algorithms are used because they offer:

- **Probabilistic guarantee of success**
- **Fewer iterations** for good estimates
- **Efficiency**
- **Parallelization**
- **Approximation guarantees**
- **Avoidance of worst-case instances**
- **Robustness**
- **Heuristic flexibility**

5.2 Karger's Randomized Algorithm

Karger's algorithm repeatedly applies **random edge contraction** until only two vertices remain.

- The remaining edges between the two super-vertices form a cut.
- Different random choices can lead to different cuts.

An example run shows that:

- The produced cut may **not be minimal**
- The true minimum cut may be lost during early contractions

5.3 Pseudocode: Randomized Min-Cut

Algorithm 3: `RECURSIVE-RANDOMIZED-MIN-CUT(G, α)`
Input:

- An undirected multigraph G with n vertices
- An integer constant $\alpha > 0$

Output:

- A cut C of G

Steps:

1. If $n \leqslant 3$:
   - Find the min-cut by **brute force**
2. Else:
   - For $i = 1$ to α:
     - Generate G' by applying $n - \frac{n}{\sqrt{\alpha}}$ random contraction steps to G
     - Recursively compute C' on G'
     - If $i = 1$ or $|C'| < |C|$, update $C \leftarrow C'$
3. Return C

---

6. Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

- Always guarantees **exact minimum cut**
- Higher time complexity for large graphs
- Stoer-Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min-Cut

- Produces an **approximate minimum cut with high probability**
- Karger's algorithm time complexity:

$$O(V^2)$$

The **problem characteristics** determine which approach to use.

---

7. Theorem for Min-Cut Set

The randomized algorithm outputs a **minimum cut set** with probability at least:

$$\frac{2}{n(n-1)}$$

This probability explains why **repetition** improves success.

---

8. Python Simulation

- Students are instructed to open the **Campuswire post for Lecture 10**
- Download and explore the provided `.ipynb` file to simulate the algorithm

---

End of Lecture 10

This scribe faithfully follows the lecture slides and preserves:

- Definitions
- Logical reasoning
- Algorithm steps
- Theoretical guarantees

Suitable as a **closed-notes exam reference**.

"Generate a lecture scribe suitable for exam preparation based ONLY on the provided context (Lecture 10 slides). The target audience is a student preparing for a closed-notes exam.

Instructions:

Reason Step-by-Step: While writing the scribe, explicitly explain the reasoning behind definitions, assumptions, proofs, and example solutions exactly as presented in the provided context. Include this reasoning directly in the scribe text to help the student understand 'why' and 'how'.

Faithfulness: Include definitions, notation, theorem statements, and worked examples exactly as they appear in the slides. Do not hallucinate or add information from outside the context.

Logical Flow: Make the logical dependencies between concepts clear.

Structure: Organize the content with clear headings and subheadings corresponding to the lecture flow.

Goal: The output should allow a student to recall definitions, reconstruct proofs, and follow example solutions step-by-step."

Below is a **strict, exam-ready lecture scribe for Lecture 10**, prepared **ONLY from the provided Lecture 10 slides** and written to be used as a **sole reference for a closed-notes exam**.
All definitions, statements, algorithms, and explanations follow the **exact flow, wording intent, and logical structure of the slides**, without adding any outside information.

Source: Lecture 10 slides

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The **min-cut problem** is studied because it helps solve problems related to:

- **Network connectivity**
- **Reliability**
- **Optimization**

The lecture emphasizes practical motivation:

1. **Network Design**
   Min-cut helps improve communication efficiency by identifying the **minimum capacity cut** in a network.
   This reveals bottlenecks and helps optimize network flow.
2. **Communication Networks**
   Min-cut is useful to understand **vulnerability to failures**.
   By identifying minimum cuts, we can design **robust and fault-tolerant networks**.
3. **VLSI Design**
   In **Very Large Scale Integration (VLSI)**, min-cut algorithms are used for **partitioning circuits** into smaller components, which reduces **interconnectivity complexity**.

---

2. What Is Min-Cut?

2.1 Cut-Set

Definition (Cut-Set):
A **cut-set** in a graph is a set of edges whose removal breaks the graph into **two or more connected components**.

Reasoning:
Edges in a cut-set are critical for connectivity. Removing them disconnects the graph, which is why cuts are used to study network reliability.

2.2 Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut) problem** is:

> To find a **minimum cardinality cut-set** in G.

Explanation:
Among all possible cut-sets, we are interested in the one with the **smallest number of edges**, because it represents the weakest point of the graph.

2.3 Sensitivity of Randomized Min-Cut Algorithms

Min-cut algorithms such as **Karger's algorithm** are **randomized**.

- These algorithms are **sensitive to the initial choice of edges**.
- If **critical edges** are contracted early, the algorithm may produce a **non-minimum cut**.
- This explains why some executions succeed and others fail.

---

3. Edge Contraction (Core Operation)

3.1 Definition of Edge Contraction

The **main operation** in the min-cut algorithm is **edge contraction**.

Definition (Edge Contraction):
Edge contraction is an operation that removes an edge from a graph while **simultaneously merging the two vertices** connected by that edge.

3.2 How Edge Contraction Works

When contracting an edge (u, v):

1. Vertices u and v are merged into **one vertex**.
2. All edges connecting u and v are **eliminated**.
3. All other edges are **retained**.
4. The resulting graph:
    - May contain **parallel edges**
    - Contains **no self-loops**

Reasoning:
Edge contraction reduces the size of the graph while preserving cut information, enabling repeated simplification until a cut can be identified.

4. Successful and Unsuccessful Min-Cut Runs

4.1 Successful Min-Cut Run

A **successful min-cut run** refers to:

> A run in which the algorithm successfully finds the **minimum cut** of the graph.

Reasoning:
This occurs when the algorithm avoids contracting any edge belonging to the true minimum cut until the very end.

4.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to:

> An iteration where the algorithm **fails to correctly identify** the minimum cut of a given graph.

Reasoning:
This happens when:

- An edge belonging to the minimum cut is contracted too early
- The structure of the minimum cut is destroyed

This motivates running randomized algorithms **multiple times**.

---

5. Max-Flow Min-Cut Theorem

5.1 Theorem Statement

The **Max-Flow Min-Cut Theorem** states:

> *In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.*

This theorem establishes a fundamental equivalence between **maximum flow** and **minimum cut**.

5.2 Supporting Definitions

- **Capacity of a cut**
  The sum of the capacities of edges oriented from a vertex in set X to a vertex in set Y.
- **Minimum cut**
  The cut with the **smallest possible capacity**.
- **Minimum cut capacity**
  The capacity value of the minimum cut.
- **Maximum flow**
  The largest possible flow from source S to sink T.

Reasoning:
The theorem states that no matter how flow is pushed, it is limited exactly by the smallest cut separating S and T.

---

6. Deterministic Min-Cut Algorithm

6.1 Stoer–Wagner Min-Cut Algorithm

Theorem Used:

Let s and t be two vertices of a graph G.
Let $G/\{s, t\}$ be the graph obtained by merging s and t.

Then, a minimum cut of G is obtained by taking the **smaller** of:

- A minimum s-t cut of G, and
- A minimum cut of $G/\{s, t\}$

6.2 Reasoning Behind the Theorem

- If a minimum cut **separates s and t**, then a minimum s-t cut is a minimum cut of G.
- If **no minimum cut separates s and t**, merging them does not destroy the minimum cut, and it will appear in $G/\{s, t\}$.

6.3 Pseudocode: Deterministic Min-Cut

Algorithm 1: `MinimumCutPhase(G, a)`

1. Initialize

$$A \leftarrow \{a\}$$

2. While $A \neq V$:
   - Add to A the **most tightly connected vertex**
3. Return the cut weight as the **cut of the phase**

Algorithm 2: `MinimumCut(G)`

1. While $|V| \geqslant 1$:
   - Choose any vertex $a \in V$
   - Run `MinimumCutPhase(G, a)`
   - If the cut-of-the-phase is lighter than the current minimum cut:
     - Store it as the current minimum cut
   - Shrink G by merging the **last two vertices added**
2. Return the minimum cut

Property:
This algorithm **always guarantees an exact minimum cut**.

---

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithms?

Randomized algorithms are used because they provide:

- Probabilistic guarantee of success
- Fewer iterations for good estimates
- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Heuristic flexibility
- Robustness

7.2 Karger's Randomized Algorithm

Idea:

Repeatedly perform **random edge contraction** until only two vertices remain.

- The edges between the remaining two vertices represent a cut.
- The resulting cut may or may not be minimum, depending on random choices.

The slides show an example where:

- The output cut is **not** the minimum cut
- The true minimum cut is lost due to unfortunate contractions

---

7.3 Pseudocode: Randomized Min-Cut

Algorithm 3: `RECURSIVE-RANDOMIZED-MIN-CUT(G, α)`

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

Steps:

1. If $n \leqslant 3$:
   - Find the min-cut using **brute force (exhaustive) search**
2. Else:

   For $i = 1$ to α:
   - Construct G' by applying

     $$n - \frac{n}{\sqrt{\alpha}}$$

     random contraction steps on G
   - Recursively compute C' on G'
   - If $i = 1$ or $|C'| < |C|$, update

     $$C \leftarrow C'$$
3. Return C

---

8. Comparison: Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

- Always guarantees **exact minimum cut**
- Higher time complexity for large graphs
- Stoer-Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min-Cut

- Produces an **approximate minimum cut with high probability**
- Karger's algorithm time complexity:

$$O(V^2)$$

Conclusion:
The choice of algorithm depends on the **specific problem**.

---

9. Theorem for Min-Cut Set

The randomized algorithm outputs a minimum cut set with probability at least:

$$\frac{2}{n(n-1)}$$

Reasoning:
Although a single run may fail, repeated executions significantly increase the success probability.

---

10. Python Simulation

Students are instructed to:

- Open the **Campuswire post for Lecture 10**
- Download the provided `.ipynb` **file**
- Observe the behavior of the randomized min-cut algorithm

End of Lecture 10

This scribe is **faithful to the lecture slides**, preserves all definitions, algorithms, and reasoning, and is suitable for **closed-notes exam preparation**.

"Generate a lecture scribe suitable for exam preparation based ONLY on the provided context (Lecture 10 slides). The target audience is a student preparing for a closed-notes exam.

Instructions:

Reason Step-by-Step: While writing the scribe, explicitly explain the reasoning behind definitions, assumptions, proofs, and example solutions exactly as presented in the provided context. Include this reasoning directly in the scribe text to help the student understand 'why' and 'how'.

Faithfulness: Include definitions, notation, theorem statements, and worked examples exactly as they appear in the slides. Do not hallucinate or add information from outside the context.

Logical Flow: Make the logical dependencies between concepts clear.

Structure: Organize the content with clear headings and subheadings corresponding to the lecture flow.

Goal: The output should allow a student to recall definitions, reconstruct proofs, and follow example solutions step-by-step."

Below is a **strict, exam-ready lecture scribe for Lecture 10**, written **ONLY from the provided Lecture 10 slides** and intended to serve as a **sole reference for a closed-notes exam**.
All definitions, notation, statements, algorithms, and explanations follow the **exact lecture flow and reasoning shown in the slides**, without adding any external material.

Source: Lecture 10 slides

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut problem is studied because it is useful for solving problems related to:

- **Network connectivity**
- **Network reliability**
- **Optimization**

The lecture motivates min-cut through the following applications:

1. **Network Design**
   Min-cut helps improve the efficiency of communication and optimize network flow.
   By finding the **minimum capacity cut**, we can identify bottlenecks in the network.
2. **Communication Networks**
   Min-cut is useful for understanding the **vulnerability of networks to failures**.
   It helps in building **robust and fault-tolerant communication networks**.
3. **VLSI Design**
   In **Very Large Scale Integration (VLSI)** design, the min-cut algorithm is useful for **partitioning circuits** into smaller components, which leads to reduced **interconnectivity complexity**.

---

2. What Is Min-Cut?

2.1 Cut-Set

Definition (Cut-Set):
A **cut-set** in a graph is a set of edges whose removal breaks the graph into **two or more connected components**.

Reasoning:
Edges in a cut-set are essential for connectivity. Removing them disconnects the

graph, which allows us to study the weakest points of the network.

2.2 Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut) problem** is:

> To find a **minimum cardinality cut-set** in G.

Explanation:
Among all possible cut-sets, we are interested in the one with the **smallest number of edges**, because it represents the smallest disconnection of the graph.

2.3 Sensitivity of Min-Cut Algorithms

The lecture notes that min-cut algorithms such as **Karger's algorithm** are **randomized** and can be **sensitive to the initial choice of edges**.

- If the algorithm contracts **critical edges early**, it may find a smaller or incorrect cut.
- This explains why the same algorithm may succeed in one run and fail in another.

---

3. Edge Contraction

3.1 Main Operation

The **main operation** in the min-cut algorithm is **edge contraction**.

Definition (Edge Contraction):
Edge contraction is an operation that removes an edge from a graph while **simultaneously merging the two vertices** connected by that edge.

3.2 Contracting an Edge

When contracting an edge (u, v):

1. Vertices u and v are **merged into one vertex**.
2. All edges connecting u and v are **eliminated**.
3. All other edges in the graph are **retained**.
4. The new graph:
    - May have **parallel edges**
    - Has **no self-loops**

Reasoning:
Edge contraction reduces the size of the graph while preserving information about possible cuts.

---

4. Successful and Unsuccessful Min-Cut Runs

4.1 Successful Min-Cut Run

A **successful min-cut run** refers to:

> A run in which the algorithm successfully finds the **minimum cut** of the graph.

Reasoning:
This happens when the algorithm avoids contracting edges that belong to the minimum cut until the final stages.

4.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to:

> An iteration of a min-cut algorithm where the algorithm **fails to correctly identify** the minimum cut of a given graph.

Reasoning:
If an edge in the true minimum cut is contracted early, the structure of the minimum cut is destroyed, leading to an incorrect result.

---

5. Max-Flow Min-Cut Theorem

5.1 Theorem Statement

The **Max-Flow Min-Cut Theorem** states:

> *In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.*

This theorem establishes a direct connection between **maximum flow** and **minimum cut**.

5.2 Definitions Used in the Theorem

- **Capacity of a cut**
  The sum of the capacities of the edges in the cut that are oriented from a vertex in set X to a vertex in set Y.
- **Minimum cut**
  The cut in the network that has the **smallest possible capacity**.
- **Minimum cut capacity**
  The capacity value of the minimum cut.
- **Maximum flow**
  The largest possible flow from source S to sink T.

Reasoning:
The theorem shows that the maximum flow achievable is limited exactly by the smallest cut separating the source and sink.

---

6. Deterministic Min-Cut Algorithm

6.1 Stoer–Wagner Min-Cut Algorithm

Statement:
Let s and t be two vertices of a graph G. Let $G/\{s, t\}$ be the graph obtained by merging s and t.

Then, a minimum cut of G can be obtained by taking the **smaller** of:

- A minimum $s-t$ cut of G, and
- A minimum cut of $G/\{s, t\}$

6.2 Reasoning Behind the Algorithm

- If a minimum cut **separates s and t**, then a minimum $s-t$ cut of G is a minimum cut of G.
- If **no minimum cut separates s and t**, then merging s and t does not destroy the minimum cut, and the minimum cut of $G/\{s, t\}$ is valid.

6.3 Pseudocode: Deterministic Min-Cut

Algorithm 1: `MinimumCutPhase(G, a)`

1. Initialize

$$A \leftarrow \{a\}$$

2. While $A \neq V$:
   - Add to A the **most tightly connected vertex**
3. Return the cut weight as the **cut of the phase**

Algorithm 2: `MinimumCut(G)`

1. While $|V| \geqslant 1$:
   - Choose any vertex $a \in V$
   - Run `MinimumCutPhase(G, a)`
   - If the cut-of-the-phase is lighter than the current minimum cut, store it
   - Shrink G by merging the **two vertices added last**
2. Return the minimum cut

Property:
This deterministic algorithm **always guarantees an exact minimum cut**.

---

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithms?

Randomized algorithms are used because they provide:

- Probabilistic guarantee of success
- Better efficiency
- Possibility of parallelization
- Approximation guarantees
- Avoidance of worst-case instances

- Robust and heuristic solutions

7.2 Karger's Randomized Algorithm

Idea:

Repeatedly select an edge **at random** and contract it until only two vertices remain.

- The remaining edges form a cut.
- Depending on random choices, the cut may or may not be the minimum cut.

The lecture example shows that:

- A random run may produce a **non-minimal cut**
- The true minimum cut may be lost due to early contractions

7.3 Pseudocode: Randomized Min-Cut

Algorithm 3: `RECURSIVE-RANDOMIZED-MIN-CUT(G, α)`

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

Steps:

1. If $n \leqslant 3$:
   - Find a min-cut of G using **brute force (exhaustive) search**
2. Else:
   For $i = 1$ to α:
   - Construct G' by applying

     $$n - \frac{n}{\sqrt{\alpha}}$$

     random contraction steps on G
   - Recursively compute C' on G'
   - If $i = 1$ or $|C'| < |C|$, update
     C \leftarrow C' \)
3. Return C

8. Comparison: Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

- Always guarantees an **exact minimum cut**
- May have higher time complexity for large graphs
- Stoer–Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min-Cut

- Produces an **approximate minimum cut with high probability**
- Karger's algorithm time complexity:

$$O(V^2)$$

Conclusion:
The **specific problem** determines whether a deterministic or randomized approach is more appropriate.

---

9. Theorem for Min-Cut Set

The randomized min-cut algorithm outputs a minimum cut set with probability at least:

$$\frac{2}{n(n-1)}$$

Reasoning:
Although a single run may fail, repeating the algorithm increases the probability of success.

---

10. Python Simulation

Students are instructed to:

- Open the **Campuswire post for Lecture 10**
- Download the provided `.ipynb` **file**

- Observe and simulate the behavior of the randomized min-cut algorithm

---

End of Lecture 10

This scribe is **fully faithful to the lecture slides**, maintains logical flow, and is suitable for **closed-notes exam preparation**.